Sumario

¿Qué es una API REST?.....6
1 Definición de API.....6
2 ¿Qué es REST?.....6
Principios de REST.....6
3 Ejemplo básico de API REST.....7
4 Analogía sencilla.....7
5 Beneficios de usar APIs REST.....8
6 Casos de uso reales.....8
Redes sociales.....8
Servicios de mapas.....8
E-commerce y pagos.....8
Aplicaciones cotidianas.....8
7 Diferencias con otros enfoques.....8
8 Conclusión.....9
Instalación de herramientas para trabajar con APIs.....9
1 Postman.....9
2 cURL.....10
3 Navegadores.....10
4 Editores de código.....11
5 Conclusión del capítulo.....11
Conceptos básicos de JSON (estructura de datos y sintaxis).....12
1 ¿Qué es JSON?.....12
2 Reglas de sintaxis.....12
3 Estructura básica.....13
a) Objetos.....13
b) Arreglos.....13
4 Combinación de objetos y arreglos.....13
5 Tipos de datos en JSON.....14
6 Validación de JSON.....14
7 Ejemplo práctico en APIs REST.....14
8 Ventajas de JSON.....15
9 Conclusión del capítulo.....15
Métodos HTTP en REST (GET, POST, PUT, DELETE, PATCH).....15
1 ¿Qué son los métodos HTTP?.....15
2 GET — Obtener información.....16
3 POST — Crear un recurso.....16
4 PUT — Actualizar un recurso completo.....16
5 PATCH — Actualizar parcialmente un recurso.....17
6 DELETE — Eliminar un recurso.....17
7 Comparación entre métodos.....18
8 Buenas prácticas en el uso de métodos HTTP.....18
9 Conclusión del capítulo.....18
Parámetros en la URL (Path Parameters y Query Parameters).....19
1 Path Parameters.....19
Uso en diferentes frameworks.....19
2 Query Parameters.....20

3 Diferencias entre Path y Query Parameters....20
4 Ejemplos prácticos con APIs....20
Ejemplo 1 — Usuarios....20
Ejemplo 2 — Pedidos....21
Ejemplo 3 — Productos....21
5 Ejemplo en Postman....21
6 Ejemplo en cURL....21
7 Buenas prácticas en el uso de parámetros....22
8 Conclusión del capítulo....22
Encabezados HTTP (Headers: Content-Type, Authorization, Accept)....22
1 ¿Qué son los headers en HTTP?....22
2 Header: Content-Type....23
3 Header: Accept....23
4 Header: Authorization....24
5 Ejemplos en herramientas....24
En Postman....24
En cURL....24
6 Otros headers comunes en APIs REST....25
7 Buenas prácticas con headers....25
8 Conclusión del capítulo....25
Códigos de estado HTTP (200, 201, 400, 404, 500, etc.)....25
1 ¿Qué son los códigos de estado HTTP?....25
2 Clasificación de los códigos HTTP....26
3 Códigos más usados en APIs REST....26
A) Respuestas exitosas (2xx)....26
B) Errores del cliente (4xx)....26
C) Errores del servidor (5xx)....27
4 Ejemplos prácticos en APIs REST....27
Ejemplo 1 — Crear un recurso....27
Ejemplo 2 — Recurso no encontrado....27
Ejemplo 3 — Error del cliente (JSON mal formado)....28
Ejemplo 4 — Sin autenticación....28
Ejemplo 5 — Eliminación exitosa sin datos....28
5 Buenas prácticas con códigos de estado....28
6 Conclusión del capítulo....29
Envío y recepción de datos en formato JSON....29
1 JSON como estándar en APIs REST....29
2 Envío de datos en formato JSON (Request)....29
Ejemplo 1 — Crear un usuario (POST)....29
Ejemplo 2 — Actualizar un producto (PUT)....30
Ejemplo 3 — Actualización parcial (PATCH)....30
3 Recepción de datos en formato JSON (Response)....30
Ejemplo 1 — Obtener un producto (GET)....30
Ejemplo 2 — Lista de recursos....31
4 Ejemplos en herramientas....31
A) Postman....31
B) cURL....31
5 Errores comunes al enviar/recibir JSON....31

6 Buenas prácticas....32
7 Conclusión del capítulo....32
Serialización y deserialización de objetos a JSON....33
1 ¿Qué es la serialización y deserialización?....33
2 ¿Por qué es importante en APIs REST?....33
3 Ejemplo conceptual....33
4 Ejemplos en distintos lenguajes....34
A) Python....34
B) JavaScript....34
C) Java (usando Jackson)....34
D) C# (.NET con System.Text.Json)....35
5 Problemas comunes en la serialización....36
6 Buenas prácticas....36
7 Conclusión del capítulo....36
Uso de JSON anidado y estructuras complejas en APIs....36
1 Introducción....36
2 JSON anidado....37
3 Arreglos de objetos....37
4 Ejemplo complejo en APIs REST....37
5 Consumo en diferentes lenguajes....38
A) Python....38
B) JavaScript....39
6 Problemas comunes con JSON anidado....39
7 Buenas prácticas con estructuras complejas....39
8 Conclusión del capítulo....39
Autenticación y autorización en APIs REST (tokens, API keys, JWT)....40
1 Introducción....40
2 Métodos de autenticación en APIs REST....40
1) API Keys....40
2) Tokens de sesión (Bearer Tokens)....40
3) JWT (JSON Web Tokens)....41
3 Flujo típico de autenticación con JWT....41
4 Ejemplo práctico....42
5 Autenticación vs Autorización....42
6 Ejemplos en herramientas....42
En Postman....42
En cURL....42
7 Buenas prácticas....42
8 Conclusión del capítulo....43
Manejo de errores en APIs REST (mensajes en JSON, estandarización)....43
1 Introducción....43
2 Principios del manejo de errores en APIs....43
3 Ejemplos de errores comunes....44
Error de validación (400 Bad Request)....44
Recurso no encontrado (404 Not Found)....44
Sin autorización (401 Unauthorized)....44
Error del servidor (500 Internal Server Error)....44
4 Estructura estandarizada para errores....45

5 Ejemplo en un flujo real....45
6 Manejo de errores en herramientas....46
En Postman....46
En cURL....46
7 Buenas prácticas para manejo de errores en APIs REST....46
8 Conclusión del capítulo....46
Documentación automática de APIs (Swagger / OpenAPI)....47
1 Introducción....47
2 ¿Qué es OpenAPI?....47
3 ¿Qué es Swagger?....47
4 Beneficios de usar Swagger / OpenAPI....48
5 Ejemplo de documentación en Swagger UI....48
6 Ejemplo de especificación OpenAPI completa (YAML)....48
7 Swagger en frameworks populares....49
8 Ejemplo práctico con FastAPI....50
9 Buenas prácticas en la documentación automática....50
10 Conclusión del capítulo....50
Versionado de APIs (v1, v2 en endpoints)....51
1 Introducción....51
2 ¿Qué es el versionado de APIs?....51
3 Estrategias comunes de versionado....51
1) Versión en la URL (la más usada)....51
2) Versión en el header....51
3) Versión en el parámetro de query....51
4 Ejemplo práctico de versionado....52
5 Migración entre versiones....52
6 Buenas prácticas en versionado....52
7 Ejemplo en frameworks....53
FastAPI (Python)....53
Express (Node.js)....53
8 Conclusión del capítulo....53
Buenas prácticas de diseño de APIs REST....54
1 Introducción....54
2 Consistencia en endpoints y rutas....54
3 Uso correcto de métodos HTTP....54
4 Uso adecuado de códigos de estado....55
5 Estructura clara en respuestas JSON....55
6 Paginación y filtros....55
7 Autenticación y seguridad....56
8 Versionado claro....56
9 Documentación automática....56
10 Estándar en mensajes de error....56
11 Estandarizar convenciones de nombres....57
12 Ejemplo de una API bien diseñada....57
13 Conclusión del capítulo....57
Testing de APIs REST (Postman, pytest, herramientas automatizadas)....58
1 Introducción....58
2 Tipos de pruebas en APIs REST....58

3 Testing con Postman....58
4 Testing con pytest (Python)....59
5 Herramientas automatizadas adicionales....60
6 Ejemplo de flujo completo de testing....60
7 Buenas prácticas en testing de APIs REST....60
8 Conclusión del capítulo....61
Consumo de APIs REST desde Python (requests, httpx)....61
1 Introducción....61
2 Consumo de APIs con requests....61
Ejemplo 1 — GET básico....61
Ejemplo 2 — POST con JSON....61
Ejemplo 3 — PUT (actualizar recurso completo)....62
Ejemplo 4 — PATCH (actualización parcial)....62
Ejemplo 5 — DELETE....62
3 Manejo de headers personalizados....62
4 Consumo de APIs con httpx....63
Ejemplo 1 — GET con httpx (síncrono)....63
Ejemplo 2 — GET con httpx (asíncrono)....63
Ejemplo 3 — POST con httpx....63
Ejemplo 4 — Headers en httpx....63
5 Manejo de errores en Python....64
6 Ejemplo práctico: consultar una lista de productos....64
7 ¿Cuándo usar requests y cuándo httpx?....64
8 Buenas prácticas en Python para consumo de APIs....65
9 Conclusión del capítulo....65
Comparación de APIs REST con otras arquitecturas (SOAP, GraphQL)....65
1 Introducción....65
2 APIs REST....65
3 APIs SOAP....66
4 APIs GraphQL....67
5 Comparación REST vs SOAP vs GraphQL....68
6 ¿Cuándo usar cada arquitectura?....68
7 Conclusión del capítulo....68

¿Qué es una API REST?

1 Definición de API

Una API (Application Programming Interface o Interfaz de Programación de Aplicaciones) es un conjunto de reglas y especificaciones que permiten que diferentes programas o sistemas se comuniquen entre sí.

Se la puede imaginar como un contrato: el proveedor de la API define qué servicios ofrece, cómo pedirlos y qué se recibe a cambio.

El cliente que la consume no necesita saber cómo está programada internamente: solo importa qué datos pide y qué datos recibe.

🔎 **Ejemplo cotidiano:**

Cuando en tu celular usás una app de clima, esta consulta a una API de un servidor meteorológico.

La app pide temperatura actual en Córdoba, y la API responde con un valor: 28°C.

👉 El usuario final no ve la API, pero gracias a ella la app funciona.

2 ¿Qué es REST?

REST significa *Representational State Transfer*. Fue propuesto en el año 2000 por Roy Fielding, uno de los autores de la especificación del protocolo HTTP, en su tesis doctoral (REST).

REST no es un programa, ni un protocolo nuevo, ni una librería. Es un estilo de arquitectura que define cómo deben diseñarse los servicios web para que sean simples, escalables y fáciles de usar.

Principios de REST

Para que una API pueda considerarse RESTful, debería seguir estas reglas:

Arquitectura Cliente-Servidor
El cliente (app móvil, navegador, software de escritorio) y el servidor (donde vive la API) están separados. El cliente pide datos y el servidor responde. Esto permite independencia: se puede cambiar la app sin modificar la API, y viceversa.

Sin estado (Stateless)
Cada petición debe contener toda la información necesaria para que el servidor la entienda. El servidor no recuerda quién sos entre una petición y otra. Ejemplo: si hacés 3 consultas a `/usuarios/1`, cada una debe incluir los datos de autenticación (ej. un token).

Uso de HTTP y sus métodos
Las APIs REST aprovechan los verbos estándar del protocolo HTTP:

- GET: obtener datos.
- POST: crear un recurso.
- PUT: actualizar un recurso.
- DELETE: eliminar un recurso.
- PATCH: actualizar parcialmente un recurso.

Recursos identificados por URLs
Cada recurso (usuario, producto, pedido) se identifica por una dirección única.

Ejemplo:
`/usuarios/1` → usuario con ID = 1.
`/productos/15` → producto con ID = 15.

Representación de recursos
Los recursos pueden representarse en distintos formatos (JSON, XML, CSV, YAML). Hoy en día, el formato más usado es JSON, por su ligereza y compatibilidad con JavaScript.

Comunicación uniforme
La API debe ser predecible y consistente. Ejemplo: si `/usuarios/1` devuelve datos en JSON, `/usuarios/2` también debería hacerlo.

3 Ejemplo básico de API REST

Supongamos que tenemos un sistema de usuarios.

Petición (Request):

```
GET https://api.ejemplo.com/usuarios/1
```

Respuesta (Response):

```
{
  "id": 1,
  "nombre": "Ana",
  "email": "ana@ejemplo.com"
}
```

👉 El cliente pidió el recurso usuario con ID 1, y la API respondió con su información en formato JSON.

4 Analogía sencilla

Imaginemos que la API REST es como un restaurante:

- El menú es la documentación de la API.
- El mozo es la API en sí: recibe el pedido del cliente y lo lleva a la cocina (el servidor).
- El cliente (navegador o app) pide un plato.
- La cocina (lógica interna del servidor) prepara la comida, pero el cliente nunca ve cómo.
- La respuesta es el plato servido.

Así funciona: pedís algo → la API lo lleva → el servidor responde → vos recibís lo que pediste.

5 Beneficios de usar APIs REST

- **Simplicidad**: se basa en reglas conocidas de HTTP.
- **Rapidez de desarrollo**: JSON es ligero y fácil de usar.
- **Interoperabilidad**: cualquier lenguaje puede consumirla (Java, Python, C#, JavaScript, Go, etc.).
- **Escalabilidad**: soporta millones de usuarios.
- **Separación de responsabilidades**: frontend y backend evolucionan de forma independiente.
- **Reutilización**: una misma API puede ser usada por una app web, una app móvil y un servicio externo.

6 Casos de uso reales

Las APIs REST están en casi todo lo que usamos día a día:

Redes sociales

- Facebook API: permite crear apps que publican posts o leen datos de perfiles.
- X API: acceder a tweets, usuarios y tendencias.

Servicios de mapas

- Google Maps API: obtener coordenadas, calcular rutas y mostrar mapas embebidos.

E-commerce y pagos

- Mercado Pago, PayPal, Stripe: procesar pagos en línea de forma segura.

Aplicaciones cotidianas

- Spotify API: permite integrar listas de reproducción, buscar canciones y obtener estadísticas de usuarios.
- APIs de clima: consultar pronósticos meteorológicos en tiempo real.

7 Diferencias con otros enfoques

- **REST vs SOAP**: SOAP es más rígido y basado en XML, mientras que REST es más liviano y flexible con JSON.
- **REST vs GraphQL**: GraphQL permite consultas más personalizadas, pero REST sigue siendo el estándar más adoptado por su simplicidad.

8 Conclusión

Una API REST es un servicio web diseñado bajo principios claros (cliente-servidor, stateless, URLs como identificadores de recursos, métodos HTTP y datos en JSON).

Permite que diferentes sistemas intercambien datos de manera eficiente, escalable y estandarizada.

Es el pilar fundamental de la mayoría de las aplicaciones modernas: desde redes sociales hasta servicios financieros y de mapas.

👉 En otras palabras: REST convirtió a Internet en una red de servicios conectados, no solo de páginas web.

Instalación de herramientas para trabajar con APIs

Para poder probar, depurar y consumir APIs REST, necesitamos un conjunto de herramientas que faciliten el trabajo tanto a nivel gráfico como en línea de comandos. En este capítulo veremos las más usadas: Postman, cURL, navegadores y editores de código.

1 Postman

Postman es una aplicación muy popular para probar APIs. Permite enviar peticiones HTTP (GET, POST, PUT, DELETE, etc.) y ver las respuestas de forma clara.

Instalación
Ingresar a la página de descargas y obtener la versión para tu sistema operativo. Instalar como cualquier programa.

Primer uso
Abrí Postman y creá una Nueva petición (New Request). Elegí el método HTTP (ejemplo: GET). Escribí la URL de la API, por ejemplo:

```
https://jsonplaceholder.typicode.com/users/1
```

Presioná Send.

👉 Vas a recibir una respuesta en JSON:

```
{
  "id": 1,
  "name": "Leanne Graham",
  "email": "Sincere@april.biz"
}
```

Ventajas de Postman

- Interfaz gráfica intuitiva.
- Guardar colecciones de peticiones.

- Manejar variables (tokens, parámetros).
- Probar autenticación con API keys o JWT.
- Generar automáticamente código en varios lenguajes (Python, Java, JS).

2 cURL

cURL es una herramienta de línea de comandos que permite enviar peticiones HTTP y ver las respuestas directamente en la terminal. Es muy usada por desarrolladores y administradores de sistemas.

Instalación en Windows

1. Descargar el ZIP desde el sitio oficial.
2. Descomprimir en `C:\\curl`.
3. Agregar `C:\\curl\\bin` al `PATH` del sistema.
4. Abrir PowerShell y ejecutar:

```
curl --version
```

Ejemplos de uso

Realizar un GET a una API:

```
curl https://jsonplaceholder.typicode.com/users/1
```

Enviar un POST con JSON:

```
curl -X POST https://jsonplaceholder.typicode.com/posts -H "Content-Type:
application/json" -d '{ "title": "Hola", "body": "Esto es un post", "userId":
1 }'
```

👉 Verás en consola la respuesta en JSON.

Ventajas de cURL

- Ligero y rápido.
- Disponible en la mayoría de sistemas operativos.
- Ideal para automatizar pruebas con scripts.

3 Navegadores

Los navegadores modernos también son útiles para trabajar con APIs REST.

Acceso directo a APIs
Si abrís una URL de una API REST en el navegador, automáticamente verás la respuesta en JSON formateada.

Ejemplo: `https://jsonplaceholder.typicode.com/users`

Herramientas de desarrollo (DevTools)
Con F12 accedés a la consola del navegador. En la pestaña Network, podés ver las peticiones que hace un sitio web. Esto es muy útil para analizar cómo una aplicación consume APIs.
Recomendación: Chrome DevTools.

Extensiones recomendadas

- JSON Formatter: mejora la visualización de JSON en el navegador.
- RESTED (Firefox): permite enviar peticiones directamente desde el navegador.

4 Editores de código

Los editores de código también ofrecen soporte para probar APIs de forma integrada.

Visual Studio Code (VS Code)

- Instalar la extensión REST Client.
- Crear un archivo `api.http` con el siguiente contenido:

```
GET https://jsonplaceholder.typicode.com/users/1
```

Presionar el botón "Send Request" encima de la petición. 👉 VS Code mostrará la respuesta en una pestaña interna.

Otros editores útiles

- JetBrains IDEs (PyCharm, IntelliJ): integran herramientas de HTTP Client.
- Sublime Text con plugins de HTTP.

5 Conclusión del capítulo

Para trabajar con APIs REST, conviene dominar tanto herramientas gráficas como de línea de comandos:

- Postman: perfecto para principiantes y para organizar pruebas de APIs.
- cURL: ideal para automatizar y trabajar en entornos de consola.
- Navegadores: útiles para probar rápidamente endpoints y analizar tráfico.
- Editores de código: permiten documentar y probar APIs directamente dentro del flujo de desarrollo.

👉 Recomendación: empezar con Postman para aprender, y luego incorporar cURL y VS Code para un trabajo más profesional y automatizado.

Conceptos básicos de JSON (estructura de datos y sintaxis)

Si querés profundizar el formato JSON de forma completa y práctica, podés ver el curso dedicado en: tutorialesprogramacionya.com/jsonya.

JSON es el formato de datos más utilizado en las APIs REST. Para poder consumir o crear APIs correctamente, primero debemos entender su estructura y sus reglas de sintaxis.

1 ¿Qué es JSON?

JSON significa JavaScript Object Notation.

- Es un formato ligero, legible por humanos y fácil de procesar por las máquinas.
- Aunque nació de JavaScript, hoy es independiente y está soportado en todos los lenguajes de programación.
- Es el formato estándar en la mayoría de las APIs REST modernas.

Ejemplo simple

```
{
  "id": 1,
  "nombre": "Ana",
  "email": "ana@ejemplo.com"
}
```

👉 Este JSON representa un objeto *usuario* con tres propiedades: `id`, `nombre` y `email`.

2 Reglas de sintaxis

Para que un archivo o mensaje JSON sea válido, debe cumplir con ciertas reglas:

- Los datos se representan como **pares clave–valor**.
- Las claves (*keys*) siempre son cadenas de texto y van entre comillas dobles (`"..."`).
- Los valores pueden ser de diferentes tipos:

- **Número**: `10`, `3.14`
- **Cadena de texto**: `"Hola"`
- **Booleano**: `true` o `false`
- **Nulo**: `null`
- **Objeto**: `{ ... }`
- **Arreglo** (lista): `[ ... ]`

- No se permiten **comentarios**.
- No debe haber **comas al final** de listas u objetos.

Ejemplo válido

```
{
  "id": 101,
  "activo": true,
  "puntos": null
}
```

Ejemplo inválido (coma extra):

```
{
  "id": 101,
  "activo": true,
}
```

3 Estructura básica

a) Objetos

Un objeto se representa con llaves `{}` y contiene pares clave–valor.

```
{
  "nombre": "Carlos",
  "edad": 30,
  "profesion": "Arquitecto"
}
```

👉 Este objeto tiene tres propiedades: `nombre`, `edad` y `profesión`.

b) Arreglos

Un arreglo (*array*) se representa con corchetes `[]` y contiene una lista ordenada de valores.

```
[
  "rojo",
  "verde",
  "azul"
]
```

👉 Este arreglo contiene tres cadenas de texto.

4 Combinación de objetos y arreglos

En JSON es muy común anidar objetos y arreglos para representar estructuras más complejas.

Ejemplo

```
{
  "id": 1,
  "nombre": "Diego",
  "telefonos": ["3511234567", "3517654321"],
  "direccion": {
    "calle": "Av. Colón",
    "numero": 1234,
    "ciudad": "Córdoba"
  }
```

```
}
```

👉 Aquí vemos:

- El usuario tiene un `id` y `nombre`.
- Una lista de `teléfonos` (arreglo de cadenas).
- Una `dirección` que en sí misma es un objeto JSON anidado.

5 Tipos de datos en JSON

- **Número**: `10`, `3.14`, `-25`
- **Cadena de texto**: `"Hola mundo"` (siempre con comillas dobles)
- **Booleano**: `true`, `false`
- **Null**: `null` (valor vacío)
- **Objeto**: `{ "clave": "valor" }`
- **Arreglo**: `[1, 2, 3]`

Ejemplo que combina todos

```
{
  "producto": "Notebook",
  "precio": 1500.75,
  "stock": true,
  "descripcion": null,
  "categorias": ["tecnología", "computadoras"],
  "proveedor": {
    "nombre": "TechStore",
    "pais": "Argentina"
  }
}
```

6 Validación de JSON

Es muy importante que el JSON esté bien formado. Existen validadores en línea como:

- jsonlint.com
- jsonformatter.curiousconcept.com

Estos sitios permiten pegar un JSON y comprobar si es válido.

👉 Ejemplo: si olvidás cerrar una llave o ponés comillas simples, el validador te avisará.

7 Ejemplo práctico en APIs REST

Cuando consumimos una API REST, normalmente el servidor responde en JSON.

Petición

```
GET https://jsonplaceholder.typicode.com/posts/1
```

Respuesta en JSON

```
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident occaecati",
  "body": "quia et suscipit..."
}
```

👉 Este JSON representa un *post* con sus propiedades: usuario, id, título y cuerpo.

8 Ventajas de JSON

- **Simplicidad**: fácil de leer y escribir.
- **Ligero**: ocupa menos espacio que XML.
- **Universal**: compatible con casi todos los lenguajes.
- **Integración con JavaScript**: nativa y directa.
- **Popularidad**: se convirtió en el formato estándar para APIs REST.

9 Conclusión del capítulo

JSON es el lenguaje común de datos en las APIs REST. Su sintaxis clara y su ligereza lo hacen perfecto para intercambiar información.

Entender cómo se construyen objetos, arreglos, valores y estructuras anidadas es esencial antes de avanzar a la práctica con APIs.

👉 A partir de ahora, cada vez que trabajemos con una API REST, veremos mensajes y respuestas en JSON.

Métodos HTTP en REST (GET, POST, PUT, DELETE, PATCH)

Cuando trabajamos con APIs REST, los recursos (usuarios, productos, pedidos, etc.) se manipulan a través de métodos HTTP. Cada método tiene un propósito claro y definido.

1 ¿Qué son los métodos HTTP?

El protocolo HTTP (*HyperText Transfer Protocol*) no solo sirve para navegar páginas web, también define acciones que los clientes pueden realizar sobre recursos en un servidor.

En REST, estas acciones se mapean directamente a los métodos HTTP:

- **GET**: obtener información.
- **POST**: crear un nuevo recurso.
- **PUT**: actualizar un recurso completo.

- **PATCH**: actualizar parcialmente un recurso.
- **DELETE**: eliminar un recurso.

👉 Esta correspondencia hace que las APIs REST sean intuitivas y predecibles.

2 GET — Obtener información

El método GET se usa para recuperar datos de un servidor.

- **Seguro**: no cambia nada en el servidor.
- **Idempotente**: si lo ejecutamos varias veces, el resultado será el mismo.

Ejemplo de petición (usando la API pública JSONPlaceholder):

```
GET https://jsonplaceholder.typicode.com/users/1
```

Respuesta en JSON:

```
{
  "id": 1,
  "name": "Leanne Graham",
  "email": "Sincere@april.biz"
}
```

👉 Sirve para listar recursos (`/users`) o para obtener uno específico (`/users/1`).

3 POST — Crear un recurso

El método POST se usa para crear un nuevo recurso en el servidor.

- **No idempotente**: cada vez que lo ejecutamos, se genera un recurso distinto.

Ejemplo con cURL (crear un post):

```
curl -X POST https://jsonplaceholder.typicode.com/posts ^
  -H "Content-Type: application/json" ^
  -d "{ \"title\": \"Hola Mundo\", \"body\": \"Este es un ejemplo\", \"userId\":
1 }"
```

Respuesta en JSON:

```
{
  "id": 101,
  "title": "Hola Mundo",
  "body": "Este es un ejemplo",
  "userId": 1
}
```

👉 El servidor devolvió un objeto con `id` generado automáticamente.

4 PUT — Actualizar un recurso completo

El método PUT reemplaza por completo un recurso existente.

- **Idempotente**: si enviamos la misma actualización varias veces, el resultado será el mismo.

Ejemplo con cURL (actualizar post con `id = 1`):

```
curl -X PUT https://jsonplaceholder.typicode.com/posts/1 ^
  -H "Content-Type: application/json" ^
  -d "{ \"id\": 1, \"title\": \"Nuevo título\", \"body\": \"Nuevo
contenido\", \"userId\": 1 }"
```

Respuesta en JSON:

```
{
  "id": 1,
  "title": "Nuevo título",
  "body": "Nuevo contenido",
  "userId": 1
}
```

👉 Observá que se sobrescribieron todos los campos del recurso.

5 PATCH — Actualizar parcialmente un recurso

El método PATCH sirve para modificar solo algunos campos de un recurso. Es muy útil cuando solo necesitamos cambiar una parte.

Ejemplo con cURL (modificar solo el título de un post):

```
curl -X PATCH https://jsonplaceholder.typicode.com/posts/1 ^
  -H "Content-Type: application/json" ^
  -d "{ \"title\": \"Título actualizado con PATCH\" }"
```

Respuesta en JSON:

```
{
  "id": 1,
  "title": "Título actualizado con PATCH",
  "body": "Nuevo contenido",
  "userId": 1
}
```

👉 A diferencia de PUT, acá no fue necesario enviar todo el objeto, solo lo que queríamos modificar.

6 DELETE — Eliminar un recurso

El método DELETE sirve para borrar un recurso del servidor.

- **Idempotente**: si eliminamos dos veces el mismo recurso, la segunda vez simplemente no encontrará nada.

Ejemplo con cURL:

```
curl -X DELETE https://jsonplaceholder.typicode.com/posts/1
```

Respuesta en JSON (servidor simulado):

```
{}
```

👉 En un servidor real, el recurso ya no estaría disponible en peticiones posteriores.

7 Comparación entre métodos

- **GET**: Acción = obtener; Idempotente = Sí; Ejemplo = `GET /usuarios/1`.
- **POST**: Acción = crear; Idempotente = No; Ejemplo = `POST /usuarios`.
- **PUT**: Acción = reemplazar; Idempotente = Sí; Ejemplo = `PUT /usuarios/1`.
- **PATCH**: Acción = modificar parte; Idempotente = No siempre; Ejemplo = `PATCH /usuarios/1`.
- **DELETE**: Acción = eliminar; Idempotente = Sí; Ejemplo = `DELETE /usuarios/1`.

8 Buenas prácticas en el uso de métodos HTTP

- Usar GET solo para obtener datos, nunca para modificar.
- Usar POST para crear y PUT/PATCH para actualizar.
- Responder siempre con códigos de estado HTTP correctos:

- `200 OK`: petición exitosa.
- `201 Created`: nuevo recurso creado.
- `400 Bad Request`: error del cliente.
- `404 Not Found`: recurso no encontrado.
- `500 Internal Server Error`: error en el servidor.

Mantener consistencia en los endpoints:

```
GET /productos            # lista de productos
POST /productos           # crear producto
GET /productos/1          # obtener producto con id=1
PUT /productos/1          # actualizar producto con id=1
DELETE /productos/1       # eliminar producto con id=1
```

9 Conclusión del capítulo

Los métodos HTTP son la base del funcionamiento de una API REST:

- **GET**: leer.
- **POST**: crear.
- **PUT**: reemplazar.
- **PATCH**: modificar parcialmente.
- **DELETE**: eliminar.

👉 Comprenderlos es fundamental, porque toda interacción con APIs REST gira en torno a estas acciones.

Parámetros en la URL (Path Parameters y Query Parameters)

Cuando trabajamos con APIs REST, muchas veces necesitamos especificar o filtrar información en la petición HTTP. Para eso usamos parámetros en la URL, que se dividen en dos tipos principales:

- **Path Parameters**: identifican un recurso específico.
- **Query Parameters**: filtran o modifican los resultados.

1 Path Parameters

Los *Path Parameters* (parámetros de ruta) se usan para identificar recursos concretos dentro de la API.

- Forman parte de la ruta misma.
- Siempre están dentro de la URL, no después del `?`.
- Generalmente se representan con un ID numérico o un identificador único.

Ejemplo

```
GET https://api.tienda.com/productos/15
```

👉 Aquí:

- `/productos` es la colección.
- `/15` es el recurso con id = 15.

Respuesta en JSON

```
{
  "id": 15,
  "nombre": "Notebook",
  "precio": 1500.00
}
```

Uso en diferentes frameworks

FastAPI (Python)

```
@app.get("/productos/{id}")
def obtener_producto(id: int):
    return {"id": id, "nombre": "Notebook"}
```

Express (Node.js)

```
app.get("/productos/:id", (req, res) => {
  const id = req.params.id;
  res.json({ id, nombre: "Notebook" });
});
```

👉 En ambos casos, el valor `15` se recibe como variable `id`.

2 Query Parameters

Los *Query Parameters* se usan para filtrar, ordenar o modificar los resultados de una petición.

- Se agregan después del signo de interrogación `?`.
- Se escriben en pares `clave=valor`.
- Si hay varios, se separan con `&`.

Ejemplo

```
GET https://api.tienda.com/productos?categoria=electronica&orden=precio_desc
```

👉 Aquí:

- `categoria=electronica` filtra productos por categoría.
- `orden=precio_desc` ordena los resultados por precio descendente.

Otro ejemplo:

```
GET https://api.tienda.com/usuarios?activo=true&limite=10
```

👉 Devuelve los primeros 10 usuarios activos.

Respuesta en JSON (lista filtrada)

```
[
  { "id": 101, "nombre": "Televisor", "precio": 2500 },
  { "id": 102, "nombre": "Notebook",  "precio": 1500 }
]
```

3 Diferencias entre Path y Query Parameters

- **Ubicación**: Path = parte de la ruta (`/productos/15`); Query = después del `?` (`/productos?categoria=ropa`).
- **Uso principal**: Path = identificar un recurso específico; Query = filtrar, ordenar, limitar resultados.
- **Cantidad**: Path = generalmente uno o pocos; Query = puede haber muchos.
- **Ejemplo**: `/usuarios/5` vs `/usuarios?activo=true&limite=10`.

👉 Regla práctica: **Path** define *qué recurso* quiero. **Query** define *cómo* lo quiero.

4 Ejemplos prácticos con APIs

Ejemplo 1 — Usuarios

```
GET /usuarios/10                          # usuario específico
GET /usuarios?activo=true&limite=5       # usuarios activos con límite
```

Ejemplo 2 — Pedidos

```
GET /usuarios/7/pedidos                  # pedidos del usuario 7
GET /pedidos?fecha=2025-09-10            # filtrar por fecha
```

Ejemplo 3 — Productos

```
GET /productos/20                        # producto específico
GET /productos?precio_min=1000&precio_max=2000  # rango de precio
```

5 Ejemplo en Postman

Postman

Path Parameter

```
URL: https://jsonplaceholder.typicode.com/posts/1
Método: GET
```

Resultado: un solo post con id = 1.

Query Parameter

```
URL: https://jsonplaceholder.typicode.com/comments?postId=1
Método: GET
```

Resultado: comentarios asociados al post con id = 1.

API utilizada: JSONPlaceholder.

6 Ejemplo en cURL

cURL

Path Parameter:

```
curl https://jsonplaceholder.typicode.com/users/2
```

Respuesta:

```
{
  "id": 2,
  "name": "Ervin Howell",
  "email": "Shanna@melissa.tv"
}
```

Query Parameter:

```
curl "https://jsonplaceholder.typicode.com/posts?userId=2"
```

Respuesta: lista de posts del usuario con id = 2.

7 Buenas prácticas en el uso de parámetros

- **Usar Path Parameters** para identificar un recurso único.

```
/productos/15     # correcto
/productos?id=15  # evitar
```

- **Usar Query Parameters** para filtros, búsquedas o configuraciones.

```
/productos?orden=precio
/usuarios?activo=true&limite=20
```

- **Mantener consistencia** en nombres: siempre en minúsculas; usar guiones medios si es necesario (`fecha-inicio`).
- **No abusar** de los Query Parameters: si se vuelve muy complejo, pensar en un endpoint específico.

8 Conclusión del capítulo

Los **Path Parameters** sirven para señalar *qué recurso* queremos. Los **Query Parameters** sirven para indicar *cómo* queremos esos datos (filtrar, ordenar, paginar). El uso correcto de ambos hace que la API sea más clara, flexible y fácil de usar.

👉 En resumen: **Path** define el recurso, **Query** define las condiciones.

Encabezados HTTP (Headers: Content-Type, Authorization, Accept)

En una API REST, los encabezados de HTTP describen metadatos sobre la petición y la respuesta: formato de datos, autenticación, negociación de contenido, cacheo y más.

1 ¿Qué son los headers en HTTP?

Los *headers* (encabezados HTTP) son pares clave–valor que viajan junto con las peticiones y respuestas en HTTP. Sirven para transmitir información adicional sobre:

- El cliente (navegador, app, script).
- El servidor (API).
- El contenido que se envía o recibe.

👉 Podés imaginarlos como el sobre de una carta: además del mensaje (*body*), necesitamos datos como remitente, destinatario, idioma, etc.

Ejemplo de petición HTTP con headers

```
GET /usuarios/1 HTTP/1.1
Host: api.ejemplo.com
Content-Type: application/json
```

```
Authorization: Bearer 12345
Accept: application/json
```

2 Header: Content-Type

¿Qué indica?
El header `Content-Type` especifica el formato de los datos que el cliente envía en el cuerpo (*body*) de la petición.

👉 Es obligatorio en peticiones `POST`, `PUT` y `PATCH` cuando enviamos datos.

Ejemplos comunes

- `application/json`: el body está en formato JSON (lo más usado en APIs REST).
- `application/xml`: el body está en XML.
- `text/plain`: texto plano.
- `multipart/form-data`: usado en formularios con archivos.

Ejemplo con JSON (crear producto)

```
POST /productos HTTP/1.1
Host: api.tienda.com
Content-Type: application/json

{
  "nombre": "Notebook",
  "precio": 1500.00
}
```

👉 El servidor sabe que los datos vienen en formato JSON gracias a `Content-Type`.

3 Header: Accept

¿Qué indica?
El header `Accept` especifica el formato de la respuesta que el cliente espera recibir del servidor.

Ejemplos

- `Accept: application/json`: quiero la respuesta en JSON.
- `Accept: application/xml`: quiero la respuesta en XML.

Ejemplo

```
GET /productos/15 HTTP/1.1
Host: api.tienda.com
Accept: application/json
```

Respuesta en JSON

```
{
  "id": 15,
  "nombre": "Notebook",
```

```
  "precio": 1500.00
}
```

👉 Si el cliente hubiera enviado `Accept: application/xml`, el servidor (si soporta XML) podría responder en ese formato.

4 Header: Authorization

¿Qué indica?
El header `Authorization` se usa para enviar credenciales de acceso. Es la forma estándar de autenticar y autorizar al cliente en una API.

Modos comunes de uso

Basic Auth — se envía usuario y contraseña codificados en Base64 (RFC 7617).

```
Authorization: Basic dXN1YXJpbzpwYXNzd29yZA==
```

Bearer Token (el más usado en APIs REST modernas) — se envía un token (por ejemplo, JWT) (RFC 6750).

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...
```

API Key — una clave secreta única para el cliente.

```
Authorization: ApiKey 12345abcdef
```

Ejemplo completo con Bearer

```
GET /usuarios/1 HTTP/1.1
Host: api.tienda.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...
Accept: application/json
```

👉 Sin este header, el servidor podría responder con 401 Unauthorized.

5 Ejemplos en herramientas

En Postman

- **Content-Type**: pestaña Headers, agregar `Content-Type: application/json`.
- **Accept**: agregar `Accept: application/json`.
- **Authorization**: pestaña Authorization, seleccionar tipo (Bearer Token, Basic Auth, API Key) y completar el campo.

En cURL

Petición POST con JSON y token:

```
curl -X POST https://api.tienda.com/productos ^
  -H "Content-Type: application/json" ^
```

```
  -H "Authorization: Bearer 12345" ^
  -H "Accept: application/json" ^
  -d "{ \"nombre\": \"Tablet\", \"precio\": 1200.00 }"
```

6 Otros headers comunes en APIs REST

- Cache-Control: define si la respuesta puede almacenarse en caché.
- User-Agent: identifica al cliente (ej. navegador o aplicación).
- Retry-After: indica al cliente cuánto esperar antes de volver a hacer una petición.
- Content-Length: tamaño del body en bytes.

7 Buenas prácticas con headers

- Incluir `Content-Type` en peticiones con body (`POST`, `PUT`, `PATCH`).
- Usar `Accept` para indicar explícitamente el formato deseado (aunque sea JSON por defecto).
- Nunca enviar credenciales en la URL; usar siempre el header `Authorization`.
- Enviar solo los headers necesarios para evitar sobrecarga en la comunicación.
- Mantener consistencia en las APIs: todas deberían soportar `Accept: application/json`.

8 Conclusión del capítulo

Los headers HTTP son piezas clave de comunicación entre cliente y servidor.

- **Content-Type** indica el formato de los datos enviados.
- **Accept** indica el formato de los datos esperados como respuesta.
- **Authorization** gestiona el acceso seguro a los recursos de la API.

👉 En resumen: los headers son los metadatos que hacen que la comunicación REST sea segura, clara y eficiente.

Códigos de estado HTTP (200, 201, 400, 404, 500, etc.)

1 ¿Qué son los códigos de estado HTTP?

Cada vez que hacemos una petición a una API REST, el servidor responde con:

- Un cuerpo (*body*): datos en JSON, XML, texto, etc.
- Un encabezado (*headers*): metadatos de la respuesta.
- Un código de estado (*status code*): número de 3 dígitos que indica el resultado de la operación.

👉 Ejemplo real de respuesta:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 1,
  "nombre": "Notebook",
  "precio": 1500
}
```

El `200 OK` nos dice que la petición fue exitosa.

2 Clasificación de los códigos HTTP

Los códigos están divididos en 5 categorías según el primer dígito:

- **1xx** (100–199): respuestas informativas.
- **2xx** (200–299): peticiones exitosas.
- **3xx** (300–399): redirecciones.
- **4xx** (400–499): errores del cliente.
- **5xx** (500–599): errores del servidor.

3 Códigos más usados en APIs REST

A) Respuestas exitosas (2xx)

200 OK

La petición fue procesada correctamente. Se usa en `GET`, `PUT`, `PATCH`, `DELETE`.

```
GET /usuarios/1  →  200 OK
```

201 Created

Indica que un nuevo recurso fue creado correctamente. Generalmente usado en `POST`.

```
POST /productos  →  201 Created
```

202 Accepted

La petición fue aceptada, pero el procesamiento aún no terminó. Se usa en procesos asíncronos.

204 No Content

La operación fue exitosa, pero no hay nada que devolver.

```
DELETE /usuarios/1  →  204 No Content
```

B) Errores del cliente (4xx)

400 Bad Request

La petición está mal formada o faltan datos. Ejemplo: enviar JSON inválido o un campo obligatorio ausente.

401 Unauthorized
El cliente no está autenticado. Falta el header `Authorization`. Más info: MDN 401.

403 Forbidden
El cliente está autenticado, pero no tiene permisos para acceder al recurso.

404 Not Found
El recurso no existe. Ejemplo: `GET /productos/999` cuando ese producto no está en la base.

405 Method Not Allowed
El método HTTP usado no está permitido en esa ruta. Ejemplo: `DELETE /usuarios` en una API que no soporta borrado.

409 Conflict
Conflicto en la petición, por ejemplo, intentar crear un usuario con un email ya existente.

429 Too Many Requests
El cliente hizo demasiadas peticiones en poco tiempo (limitación por seguridad).

C) Errores del servidor (5xx)

500 Internal Server Error
Error genérico del servidor. Indica que algo falló del lado del backend.

502 Bad Gateway
El servidor recibió una respuesta inválida de otro servicio al que consultó.

503 Service Unavailable
El servidor está temporalmente fuera de servicio (mantenimiento o sobrecarga).

504 Gateway Timeout
El servidor no recibió respuesta a tiempo de otro servicio.

4 Ejemplos prácticos en APIs REST

Ejemplo 1 — Crear un recurso

```
POST /usuarios
Body: { "nombre": "Ana", "email": "ana@ejemplo.com" }

Respuesta:
201 Created
{
  "id": 5,
  "nombre": "Ana",
  "email": "ana@ejemplo.com"
}
```

Ejemplo 2 — Recurso no encontrado

```
GET /usuarios/99
```

```
Respuesta:
404 Not Found
{
  "error": "Usuario no existe"
}
```

Ejemplo 3 — Error del cliente (JSON mal formado)

```
POST /productos
Body: { "nombre": "Tablet", "precio": 1200, }   <-- coma extra

Respuesta:
400 Bad Request
{
  "error": "JSON inválido"
}
```

Ejemplo 4 — Sin autenticación

```
GET /pedidos

Respuesta:
401 Unauthorized
{
  "error": "Se requiere token de autorización"
}
```

Ejemplo 5 — Eliminación exitosa sin datos

```
DELETE /productos/10

Respuesta:
204 No Content
```

5 Buenas prácticas con códigos de estado

- Usar siempre el código correcto para que el cliente entienda qué pasó. No devolver `200 OK` cuando hubo un error.
- Acompañar el código con un mensaje claro en JSON.

Ejemplo

```
{ "error": "El campo 'email' es obligatorio" }
```

- Usar `201 Created` cuando se crea un recurso nuevo.
- Usar `204 No Content` en `DELETE` para evitar devolver datos innecesarios.
- Evitar mensajes genéricos con `500 Internal Server Error`: siempre que sea posible, dar más detalle (sin exponer información sensible).

6 Conclusión del capítulo

Los códigos de estado HTTP son fundamentales para la comunicación entre cliente y servidor en una API REST:

- **2xx**: éxito (`200 OK`, `201 Created`, `204 No Content`).
- **4xx**: error del cliente (`400 Bad Request`, `401 Unauthorized`, `404 Not Found`).
- **5xx**: error del servidor (`500 Internal Server Error`, `503 Service Unavailable`).

👉 Usar correctamente los códigos hace que una API sea clara, profesional y fácil de consumir.

Envío y recepción de datos en formato JSON

1 JSON como estándar en APIs REST

En la mayoría de las APIs modernas, el formato de intercambio de datos por defecto es JSON.

- Cuando el cliente envía datos al servidor (crear/actualizar recursos), usa JSON en el body.
- Cuando el servidor responde, devuelve datos en JSON.

👉 Esto garantiza interoperabilidad: cualquier lenguaje de programación puede generar o leer JSON fácilmente.

2 Envío de datos en formato JSON (Request)

Cuando enviamos datos a una API (por ejemplo en `POST`, `PUT` o `PATCH`), los datos van en el cuerpo de la petición (*body*) en formato JSON.

Reglas básicas

- Incluir el header `Content-Type: application/json` para indicar que estamos enviando JSON. Ver especificación de HTTP.
- Incluir los datos en el body de la petición, bien formados como objeto JSON.

Ejemplo 1 — Crear un usuario (POST)

Petición HTTP

```
POST /usuarios HTTP/1.1
Host: api.tienda.com
Content-Type: application/json
Accept: application/json

{
  "nombre": "Carlos",
  "email": "carlos@ejemplo.com",
  "edad": 28
}
```

Respuesta

```
201 Created
{
  "id": 101,
  "nombre": "Carlos",
  "email": "carlos@ejemplo.com",
  "edad": 28
}
```

👉 El servidor devuelve un nuevo objeto con `id` generado automáticamente.

Ejemplo 2 — Actualizar un producto (PUT)

```
PUT /productos/10 HTTP/1.1
Content-Type: application/json
Accept: application/json

{
  "nombre": "Notebook Gamer",
  "precio": 2500.00,
  "stock": true
}
```

👉 El servidor reemplaza todos los campos del producto con `id = 10`.

Ejemplo 3 — Actualización parcial (PATCH)

```
PATCH /productos/10 HTTP/1.1
Content-Type: application/json
Accept: application/json

{
  "precio": 2200.00
}
```

👉 Solo se modifica el precio; el resto de atributos se mantiene igual.

3 Recepción de datos en formato JSON (Response)

Cuando una API REST responde, casi siempre lo hace en JSON.

Ejemplo 1 — Obtener un producto (GET)

```
GET /productos/10 HTTP/1.1
Accept: application/json
```

Respuesta

```
{
  "id": 10,
  "nombre": "Notebook Gamer",
  "precio": 2200.00,
  "stock": true
}
```

👉 El cliente ahora puede usar estos datos en su aplicación.

Ejemplo 2 — Lista de recursos

```
GET /usuarios HTTP/1.1
Accept: application/json
```

Respuesta

```
[
  { "id": 1, "nombre": "Ana",  "email": "ana@ejemplo.com" },
  { "id": 2, "nombre": "Luis", "email": "luis@ejemplo.com" }
]
```

👉 El servidor devuelve un arreglo de objetos JSON.

4 Ejemplos en herramientas

A) Postman

- Seleccionás el método `POST` o `PUT`.
- En la pestaña *Body* → opción *raw* → seleccionás **JSON**.
- Pegás el JSON en el área de texto.

Ejemplo

```
{
  "titulo": "Nuevo curso",
  "precio": 1200
}
```

En Headers, Postman agrega automáticamente: `Content-Type: application/json`.

B) cURL

Crear un recurso con POST

```
curl -X POST https://api.tienda.com/productos ^
  -H "Content-Type: application/json" ^
  -d "{ \"nombre\": \"Tablet\", \"precio\": 900.50 }"
```

Obtener un recurso con GET

```
curl -X GET https://api.tienda.com/productos/1 ^
  -H "Accept: application/json"
```

5 Errores comunes al enviar/recibir JSON

Olvidar el Content-Type
El servidor no sabe que los datos son JSON.

Formato inválido
Comas de más, comillas simples en vez de dobles, etc.

Ejemplo incorrecto

```
{ 'nombre': 'Ana' }
```

Ejemplo correcto

```
{ "nombre": "Ana" }
```

Confundir Path Parameters con JSON
Incorrecto: enviar `id` en el body cuando debería estar en la ruta (`/usuarios/5`).

Falta de Accept en la petición
El cliente pide JSON pero no lo indica, y el servidor puede responder en otro formato.

6 Buenas prácticas

- Siempre incluir `Content-Type: application/json` cuando enviamos datos en el body.
- Siempre incluir `Accept: application/json` para asegurarnos de recibir datos en JSON.
- Validar el JSON antes de enviarlo (se pueden usar validadores en línea como jsonlint.com).
- Responder con mensajes JSON estandarizados en caso de error:

```
{ "error": "El campo 'email' es obligatorio" }
```

- Usar estructuras claras y consistentes en todas las respuestas.

7 Conclusión del capítulo

El JSON es el formato universal para enviar y recibir datos en APIs REST.

- **Enviar datos**: incluir `Content-Type: application/json` y un body bien formado.
- **Recibir datos**: incluir `Accept: application/json` para obtener la respuesta en JSON.

El correcto manejo de JSON garantiza que la API sea comprensible, segura y confiable.

👉 En resumen: JSON es el idioma que habla REST.

Serialización y deserialización de objetos a JSON

1 ¿Qué es la serialización y deserialización?

Serialización: proceso de convertir un objeto de un lenguaje de programación (por ejemplo, una clase en Python o Java) a un formato de texto plano, en este caso JSON, para enviarlo o almacenarlo.

Deserialización: proceso inverso: convertir un JSON recibido en un objeto del lenguaje para poder manipularlo en el código.

👉 Podés pensar la serialización como "empaquetar" y la deserialización como "desempaquetar".

Esquema conceptual

```
Objeto (Python, Java, JS, etc.)
       ── serialización ──▶ JSON (texto plano, formato de intercambio)
       ◀─ deserialización ── Objeto en el programa
```

2 ¿Por qué es importante en APIs REST?

En una API REST:

- Cuando enviamos datos al servidor, necesitamos serializarlos a JSON para que el servidor los entienda.
- Cuando recibimos datos del servidor, debemos deserializarlos para usarlos como objetos en nuestro programa.

👉 Sin serialización/deserialización no habría forma práctica de comunicar objetos entre sistemas en distintos lenguajes.

3 Ejemplo conceptual

Imaginemos un objeto `Producto` en un lenguaje de programación:

```
Producto {
  id: 1,
  nombre: "Notebook",
  precio: 1500.0
}
```

Serialización a JSON

```
{
  "id": 1,
  "nombre": "Notebook",
  "precio": 1500.0
}
```

Deserialización desde JSON

El JSON recibido del servidor se convierte otra vez en un objeto `Producto` dentro del programa.

4 Ejemplos en distintos lenguajes

A) Python

```
import json

# Objeto (diccionario)
producto = {
    "id": 1,
    "nombre": "Notebook",
    "precio": 1500.0
}

# Serialización (objeto → JSON)
json_str = json.dumps(producto, ensure_ascii=False, indent=4)
print("JSON generado:\n", json_str)

# Deserialización (JSON → objeto)
json_recibido = '{"id": 2, "nombre": "Tablet", "precio": 900.5}'
producto_obj = json.loads(json_recibido)
print("Objeto Python:", producto_obj["nombre"], producto_obj["precio"])
```

👉 `json.dumps()` convierte objeto a JSON. `json.loads()` convierte JSON a objeto (diccionario).

B) JavaScript

```
// Objeto JavaScript
const producto = {
  id: 1,
  nombre: "Notebook",
  precio: 1500.0
};

// Serialización (objeto → JSON)
const jsonStr = JSON.stringify(producto, null, 2);
console.log("JSON generado:\n", jsonStr);

// Deserialización (JSON → objeto)
const jsonRecibido = '{"id":2,"nombre":"Tablet","precio":900.5}';
const productoObj = JSON.parse(jsonRecibido);
console.log("Objeto JS:", productoObj.nombre, productoObj.precio);
```

👉 `JSON.stringify()` serializa. `JSON.parse()` deserializa. Referencia: MDN JSON.

C) Java (usando Jackson)

```
import com.fasterxml.jackson.databind.ObjectMapper;

class Producto {
    public int id;
```

```
    public String nombre;
    public double precio;
}

public class Main {
    public static void main(String[] args) throws Exception {
        ObjectMapper mapper = new ObjectMapper();

        // Objeto Java
        Producto producto = new Producto();
        producto.id = 1;
        producto.nombre = "Notebook";
        producto.precio = 1500.0;

        // Serialización (objeto → JSON)
        String jsonStr = mapper.writeValueAsString(producto);
        System.out.println("JSON generado:\n" + jsonStr);

        // Deserialización (JSON → objeto)
        String jsonRecibido =
"{\"id\":2,\"nombre\":\"Tablet\",\"precio\":900.5}";
        Producto prod2 = mapper.readValue(jsonRecibido, Producto.class);
        System.out.println("Objeto Java: " + prod2.nombre + " $" +
prod2.precio);
    }
}
```

👉 `writeValueAsString()` convierte objeto a JSON. `readValue()` convierte JSON a objeto. Librería: Jackson.

D) C# (.NET con System.Text.Json)

```
using System;
using System.Text.Json;

public class Producto {
    public int Id { get; set; }
    public string Nombre { get; set; }
    public double Precio { get; set; }
}

class Program {
    static void Main() {
        // Objeto C#
        Producto producto = new Producto { Id = 1, Nombre = "Notebook", Precio =
1500.0 };

        // Serialización (objeto → JSON)
        string jsonStr = JsonSerializer.Serialize(producto, new
JsonSerializerOptions { WriteIndented = true });
        Console.WriteLine("JSON generado:\n" + jsonStr);

        // Deserialización (JSON → objeto)
        string jsonRecibido =
"{\"Id\":2,\"Nombre\":\"Tablet\",\"Precio\":900.5}";
        Producto prod2 = JsonSerializer.Deserialize(jsonRecibido);
        Console.WriteLine($"Objeto C#: {prod2.Nombre} - {prod2.Precio}");
```

```
    }
}
```

👉 `JsonSerializer.Serialize()` serializa. `JsonSerializer.Deserialize()` deserializa. Docs: System.Text.Json.

5 Problemas comunes en la serialización

- **Campos faltantes o extra**: si el JSON no tiene todos los atributos, algunos lenguajes lanzan error o ignoran valores.
- **Tipos incorrectos**: `"precio": "1500"` (texto) en vez de `1500` (número).
- **Codificación de caracteres**: es importante usar UTF-8 para soportar acentos y caracteres especiales.
- **Estructuras anidadas complejas**: listas dentro de objetos requieren una deserialización más cuidada.

6 Buenas prácticas

- Validar siempre el JSON antes de procesarlo.
- Documentar los campos obligatorios y opcionales.
- Usar librerías estándar para serializar/deserializar (no reinventar la rueda).
- Evitar depender del orden de los atributos en el JSON.
- Manejar errores de forma clara:

```
{ "error": "Formato de JSON inválido" }
```

7 Conclusión del capítulo

Serialización = convertir objetos a JSON para enviar a la API.

Deserialización = convertir JSON recibido en objetos para manipular en el código.

Todos los lenguajes modernos tienen librerías que lo hacen automáticamente. Es la pieza que permite que un objeto en Python pueda comunicarse con un objeto en JavaScript a través de JSON.

👉 En resumen: sin serialización y deserialización, las APIs REST no podrían hablar el mismo idioma.

Uso de JSON anidado y estructuras complejas en APIs

1 Introducción

En la vida real, los datos que manejan las aplicaciones rara vez son simples. Por ejemplo:

- Un usuario puede tener varias direcciones.
- Un pedido puede incluir muchos productos.
- Un comentario puede incluir información del autor y respuestas.

👉 Para representar estas relaciones, las APIs REST usan JSON anidado y estructuras complejas.

2 JSON anidado

Un JSON anidado es aquel donde un valor de un objeto no es solo un número o texto, sino otro objeto o un arreglo de objetos.

Ejemplo básico (usuario con dirección)

```
{
  "id": 1,
  "nombre": "Carlos",
  "email": "carlos@ejemplo.com",
  "direccion": {
    "calle": "Av. Colón",
    "numero": 1234,
    "ciudad": "Córdoba",
    "pais": "Argentina"
  }
}
```

👉 Aquí el campo `direccion` es un objeto JSON anidado dentro del usuario.

3 Arreglos de objetos

Muchas veces un recurso está compuesto por una lista de objetos.

Ejemplo (usuario con múltiples teléfonos)

```
{
  "id": 1,
  "nombre": "Ana",
  "telefonos": [
    { "tipo": "móvil", "numero": "3511234567" },
    { "tipo": "fijo",  "numero": "3517654321" }
  ]
}
```

👉 El campo `telefonos` es un arreglo de objetos, cada uno con `tipo` y `numero`.

4 Ejemplo complejo en APIs REST

Supongamos una API de pedidos en un e-commerce.

Petición (crear un pedido)

```
POST /pedidos
Content-Type: application/json
Accept: application/json
```

```
{
  "usuario": {
    "id": 5,
    "nombre": "Laura"
  },
  "productos": [
    { "id": 101, "nombre": "Notebook", "precio": 1500.0, "cantidad": 1 },
    { "id": 202, "nombre": "Mouse inalámbrico", "precio": 25.0, "cantidad": 2 }
  ],
  "direccion_envio": {
    "calle": "San Martín",
    "numero": 500,
    "ciudad": "Córdoba",
    "pais": "Argentina"
  },
  "estado": "pendiente"
}
```

Respuesta del servidor

```
{
  "pedido_id": 9001,
  "usuario": { "id": 5, "nombre": "Laura" },
  "productos": [
    { "id": 101, "nombre": "Notebook", "precio": 1500.0, "cantidad": 1,
"subtotal": 1500.0 },
    { "id": 202, "nombre": "Mouse inalámbrico", "precio": 25.0, "cantidad": 2,
"subtotal": 50.0 }
  ],
  "total": 1550.0,
  "direccion_envio": {
    "calle": "San Martín",
    "numero": 500,
    "ciudad": "Córdoba",
    "pais": "Argentina"
  },
  "estado": "pendiente",
  "fecha_creacion": "2025-09-10T12:30:00Z"
}
```

👉 Aquí tenemos: un objeto `usuario`, un arreglo de objetos `productos`, otro objeto `direccion_envio` y atributos simples como `total` y `estado`.

5 Consumo en diferentes lenguajes

A) Python

```
import requests  # https://requests.readthedocs.io/

resp = requests.get("https://api.ejemplo.com/pedidos/9001")
pedido = resp.json()

print("Pedido ID:", pedido["pedido_id"])
print("Usuario:", pedido["usuario"]["nombre"])
for prod in pedido["productos"]:
```

```
    print(" -", prod["nombre"], "x", prod["cantidad"], "=", prod["subtotal"])
```

👉 Se accede a objetos anidados con `["campo"]` y a listas con `for`. Librería requests.

B) JavaScript

```
fetch("https://api.ejemplo.com/pedidos/9001") // API fetch:
https://developer.mozilla.org/docs/Web/API/fetch
  .then(res => res.json())
  .then(pedido => {
    console.log("Pedido ID:", pedido.pedido_id);
    console.log("Usuario:", pedido.usuario.nombre);
    pedido.productos.forEach(p => {
      console.log(` - ${p.nombre} x${p.cantidad} = ${p.subtotal}`);
    });
  })
  .catch(err => console.error("Error:", err));
```

👉 En JavaScript es muy natural manejar estructuras anidadas, porque JSON proviene de JavaScript y el API fetch lo convierte fácilmente.

6 Problemas comunes con JSON anidado

- **Estructuras demasiado profundas**: objetos dentro de objetos dentro de objetos → difícil de leer y procesar. Solución: dividir en recursos más simples con endpoints separados.
- **Datos repetidos**: evitar duplicar información innecesaria en estructuras anidadas.
- **Errores de deserialización**: si un cliente espera un objeto, pero la API devuelve un arreglo, puede fallar el parseo.

7 Buenas prácticas con estructuras complejas

- **Mantener un nivel razonable de anidación**: idealmente no más de 3 niveles de profundidad.
- **Usar arreglos cuando corresponda**: por ejemplo, `productos` debería ser siempre un arreglo, aunque tenga 1 solo elemento.
- **Estandarizar respuestas**: usar la misma estructura para todas las respuestas relacionadas.
- **Documentar bien la API**: un JSON complejo sin documentación genera confusión.

8 Conclusión del capítulo

Los JSON anidados permiten representar relaciones entre entidades (usuarios con direcciones, pedidos con productos, etc.). Son indispensables en APIs REST reales porque los datos del mundo real son jerárquicos y relacionados.

Es fundamental consumirlos correctamente en el cliente y mantener buenas prácticas de diseño.

👉 En resumen: JSON anidado es la forma en que REST modela la complejidad del mundo real.

Autenticación y autorización en APIs REST (tokens, API keys, JWT)

1 Introducción

Cuando usamos una API REST, muchas veces necesitamos proteger el acceso:

- No cualquiera debería poder leer datos sensibles (usuarios, pagos, etc.).
- No cualquiera debería poder modificar o eliminar recursos.

👉 Para eso existen dos conceptos clave:

- **Autenticación**: comprobar quién es el cliente (identidad).
- **Autorización**: comprobar qué puede hacer ese cliente (permisos).

Ejemplo

Cuando entrás a tu cuenta de correo, primero te autenticás con usuario y contraseña. Después, según tu rol, podés estar autorizado a leer tus emails pero no los de otra persona.

2 Métodos de autenticación en APIs REST

Existen varios enfoques. Los más comunes son:

1) API Keys

Una *API Key* es una cadena única que identifica al cliente. Se entrega al usuario cuando se registra en el servicio. Se envía en el header o como parámetro en la URL.

Ejemplo en header

```
GET /datos HTTP/1.1
Host: api.ejemplo.com
Authorization: ApiKey 12345abcdef
```

Ejemplo en cURL

```
curl -H "Authorization: ApiKey 12345abcdef" https://api.ejemplo.com/datos
```

👉 Son fáciles de usar pero no tan seguras como otros métodos, porque pueden ser robadas si no se transmiten por HTTPS.

2) Tokens de sesión (Bearer Tokens)

Un token es un identificador generado por el servidor al momento de autenticarse (normalmente con usuario + contraseña). El cliente debe incluir este token en cada petición, en el header `Authorization`.

```
GET /usuarios/1 HTTP/1.1
```

```
Host: api.ejemplo.com
Authorization: Bearer abcdef123456
```

👉 El prefijo *Bearer* significa portador, indicando que quien tenga el token puede acceder. Esquema definido en RFC 6750.

3) JWT (JSON Web Tokens)

Un JWT (*JSON Web Token*) es un estándar muy popular para manejar autenticación y autorización en APIs REST.

Características

- Es un token en formato JSON codificado en Base64.
- Está compuesto de 3 partes separadas por puntos: `header.payload.signature`.
- Incluye *claims* (información del usuario, roles, permisos).
- El servidor no guarda sesión: toda la info viaja en el token — ideal para APIs *stateless*.

Ejemplo de JWT


```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJ1c2VyaWQiOjEsInJvbCI6ImFkbWluIn0.
h5tG1zX4V4hKc-tZDfPi7dyW4QaU57aV6lzPt-8L9gk
```


- **Header**: tipo de token y algoritmo de firma.
- **Payload**: datos (`userId`, rol, expiración).
- **Signature**: garantiza que no fue modificado.

Payload decodificado

```
{
  "userId": 1,
  "rol": "admin",
  "exp": 1715380000
}
```

👉 El servidor puede verificar la firma sin necesidad de guardar estado; esto hace a JWT muy eficiente en arquitecturas distribuidas.

3 Flujo típico de autenticación con JWT

1. El cliente envía usuario + contraseña a `/login`.
2. El servidor valida credenciales y devuelve un JWT.
3. El cliente guarda el token (por ejemplo, en localStorage si es web).
4. En cada petición, el cliente envía el token en el header: `Authorization: Bearer <token>`.
5. El servidor valida la firma del token y revisa los permisos.

4 Ejemplo práctico

Petición de login

```
POST /login
Content-Type: application/json

{
  "email": "ana@ejemplo.com",
  "password": "12345"
}
```

Respuesta con token JWT

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR..."
}
```

Uso del token en una petición protegida

```
GET /pedidos/9001
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...
```

👉 Si el token es válido, el servidor devuelve el pedido. Si el token falta o está vencido, responde con 401 Unauthorized.

5 Autenticación vs Autorización

- **Autenticación**: validar identidad. Ejemplo: ¿es realmente Ana? (token válido).
- **Autorización**: validar permisos. Ejemplo: ¿Ana puede ver pedidos de otros usuarios o solo los suyos?

En muchas APIs, después de autenticar al usuario, se incluyen roles en el token (admin, editor, usuario) para manejar la autorización.

6 Ejemplos en herramientas

En Postman

- Pestaña *Authorization*: Tipo: API Key, Bearer Token, o Basic Auth.
- Se agrega automáticamente al header.

En cURL

```
curl -X GET https://api.ejemplo.com/usuarios ^
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR..."
```

7 Buenas prácticas

- Siempre usar HTTPS para evitar robo de tokens en tránsito.

- Nunca guardar contraseñas en texto plano: usar hashing (bcrypt, Argon2).
- Configurar expiración de tokens (ejemplo: 15 minutos) y renovar con *refresh token*.
- Evitar poner tokens en la URL; enviarlos solo en headers.
- Aplicar autorización por roles y, si es necesario, por permisos granulares.

8 Conclusión del capítulo

La **autenticación** asegura que el cliente es quien dice ser; la **autorización** controla qué acciones puede realizar ese cliente.

Métodos comunes: API Keys, Bearer Tokens, **JWT**.

Los JWT se han convertido en el estándar de facto en APIs REST por ser seguros, livianos y *stateless*.

👉 En resumen: sin autenticación y autorización, una API REST estaría expuesta a cualquiera y no sería segura.

Manejo de errores en APIs REST (mensajes en JSON, estandarización)

1 Introducción

En cualquier API REST, los errores son inevitables:

- El cliente puede enviar datos mal formados.
- Puede intentar acceder a un recurso que no existe.
- El servidor puede fallar por problemas internos.

👉 Lo importante no es solo detectar los errores, sino comunicarlos de manera clara, consistente y útil al cliente.

2 Principios del manejo de errores en APIs

- Usar siempre códigos HTTP correctos que indiquen el tipo de error.
- Devolver un cuerpo en formato JSON: no solo un código, sino un mensaje descriptivo.
- Estandarizar el formato de respuesta de error: todas las respuestas de error deben seguir la misma estructura.
- No exponer información sensible: nunca mostrar contraseñas, rutas internas del servidor ni stack traces completos.

3 Ejemplos de errores comunes

Error de validación (400 Bad Request)

```
POST /usuarios
Content-Type: application/json

{
  "nombre": "",
  "email": "correo_invalido"
}
```

Respuesta JSON

```
{
  "error": "Bad Request",
  "mensaje": "El campo 'nombre' es obligatorio y el email no es válido",
  "codigo": 400
}
```

Recurso no encontrado (404 Not Found)

```
GET /productos/9999
```

Respuesta JSON

```
{
  "error": "Not Found",
  "mensaje": "El producto con id 9999 no existe",
  "codigo": 404
}
```

Sin autorización (401 Unauthorized)

```
GET /pedidos

(sin enviar token de acceso)
```

Respuesta JSON

```
{
  "error": "Unauthorized",
  "mensaje": "Se requiere autenticación para acceder a este recurso",
  "codigo": 401
}
```

Error del servidor (500 Internal Server Error)

```
GET /usuarios
```

Respuesta JSON

```
{
  "error": "Internal Server Error",
```

```
  "mensaje": "Ocurrió un error inesperado en el servidor. Intente más tarde.",
  "codigo": 500
}
```

👉 Nota: el cliente no debería ver detalles internos como *NullPointerException* en `UserService`.

4 Estructura estandarizada para errores

Una buena práctica es definir un formato de error consistente.

Ejemplo recomendado

```
{
  "error": "Bad Request",
  "mensaje": "El campo 'email' es obligatorio",
  "codigo": 400,
  "detalles": [
    { "campo": "email", "problema": "formato inválido" }
  ],
  "timestamp": "2025-09-10T12:30:00Z",
  "ruta": "/usuarios"
}
```

Campos sugeridos

- `error`: tipo genérico (ej. Bad Request, Not Found).
- `mensaje`: descripción clara y legible.
- `codigo`: número del status HTTP.
- `detalles`: errores específicos (útil en validaciones).
- `timestamp`: fecha/hora en que ocurrió el error.
- `ruta`: endpoint que produjo el error.

5 Ejemplo en un flujo real

Petición

```
POST /productos
Content-Type: application/json

{
  "precio": -50
}
```

Respuesta

```
{
  "error": "Bad Request",
  "mensaje": "Faltan campos obligatorios y el precio debe ser mayor que 0",
  "codigo": 400,
  "detalles": [
    { "campo": "nombre", "problema": "obligatorio" },
    { "campo": "precio", "problema": "valor negativo" }
  ],
```

```
  "timestamp": "2025-09-10T14:00:00Z",
  "ruta": "/productos"
}
```

👉 Con esta respuesta, el cliente sabe exactamente qué corregir.

6 Manejo de errores en herramientas

En Postman

- Si la API devuelve un error en JSON, Postman lo muestra formateado.
- Se pueden crear tests automáticos para validar que los errores devuelvan el status correcto (`pm.response.code === 400`).

En cURL

```
curl -X POST https://api.tienda.com/productos ^
  -H "Content-Type: application/json" ^
  -d "{ \"precio\": -50 }"
```

Respuesta en consola

```
{
  "error": "Bad Request",
  "mensaje": "El precio debe ser mayor que 0",
  "codigo": 400
}
```

7 Buenas prácticas para manejo de errores en APIs REST

- No devolver solo un número: `400`. Incluir JSON con detalles del error.
- Usar siempre el mismo formato de error: el cliente sabe qué esperar y cómo parsearlo.
- Evitar mensajes ambiguos: mejor `"El campo 'email' es obligatorio"` que `"Error en la petición"`.
- No exponer detalles internos: los logs completos deben guardarse en el servidor, no enviarse al cliente.
- Incluir un `timestamp` y la `ruta`: ayuda en la depuración y trazabilidad.
- Documentar los posibles errores de cada endpoint: ejemplo, `/usuarios` puede devolver 400, 401, 404, 500.

8 Conclusión del capítulo

El manejo de errores en APIs REST es tan importante como el manejo de respuestas exitosas:

- Un buen diseño de errores mejora la experiencia de los clientes de la API.
- Los errores deben ser claros, consistentes y útiles.
- La estandarización es clave: misma estructura de error en todos los endpoints.

👉 En resumen: los errores bien diseñados hacen que tu API sea profesional y fácil de usar.

Documentación automática de APIs (Swagger / OpenAPI)

1 Introducción

La documentación es uno de los elementos más importantes en una API REST.

- Una API mal documentada es difícil de entender y casi imposible de usar por terceros.
- Una API bien documentada permite que cualquier desarrollador pueda integrarse fácilmente.

👉 El estándar actual para documentar APIs REST es la OpenAPI Specification (OAS), y la herramienta más popular para visualizarla e interactuar con ella es Swagger UI.

2 ¿Qué es OpenAPI?

OpenAPI es una especificación estandarizada para describir APIs REST. Permite definir:

- Rutas (endpoints).
- Métodos HTTP (GET, POST, etc.).
- Parámetros (Path, Query, Header).
- Estructuras de request y response en JSON.
- Códigos de estado.
- Autenticación (API keys, JWT, OAuth2).

👉 Se escribe en YAML o JSON, y sirve como contrato entre el servidor y los clientes.

Ejemplo simple en YAML

```
openapi: 3.0.0
info:
  title: API de Productos
  version: 1.0.0
paths:
  /productos:
    get:
      summary: Obtener todos los productos
      responses:
        '200':
          description: Lista de productos
```

3 ¿Qué es Swagger?

Swagger es un ecosistema de herramientas que trabajan con OpenAPI.

- Swagger UI: genera una interfaz web interactiva para probar la API.
- Swagger Editor: permite escribir y validar especificaciones OpenAPI.

- Swagger Codegen: genera clientes y servidores automáticamente a partir de la especificación.

👉 Hoy en día, Swagger UI es lo que los desarrolladores ven más seguido: una página web con la lista de endpoints donde pueden probar la API en tiempo real.

4 Beneficios de usar Swagger / OpenAPI

- **Documentación automática**: se genera desde el código o la especificación.
- **Interactividad**: los desarrolladores pueden ejecutar peticiones directamente desde la doc.
- **Estandarización**: todas las APIs siguen la misma estructura.
- **Generación de SDKs**: se pueden generar clientes en Java, Python, C#, etc., automáticamente.
- **Mejora la comunicación**: es un contrato claro entre backend y frontend.

5 Ejemplo de documentación en Swagger UI

Supongamos que tenemos una API con este endpoint:

```
GET /productos
```

En Swagger UI se vería así:

- **Método**: GET
- **Ruta**: /productos
- **Descripción**: Devuelve todos los productos.
- **Respuesta 200**:

```
[
  { "id": 1, "nombre": "Notebook", "precio": 1500 },
  { "id": 2, "nombre": "Tablet", "precio": 900 }
]
```

👉 El desarrollador puede presionar *Try it out*, ejecutar la petición y ver la respuesta directamente.

6 Ejemplo de especificación OpenAPI completa (YAML)

```
openapi: 3.0.0
info:
  title: API de Usuarios
  version: 1.0.0
paths:
  /usuarios:
    get:
      summary: Listar usuarios
      responses:
        '200':
          description: Lista de usuarios
          content:
            application/json:
              schema:
```

```
                type: array
                items:
                  $ref: '#/components/schemas/Usuario'
    post:
      summary: Crear un nuevo usuario
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Usuario'
      responses:
        '201':
          description: Usuario creado
  /usuarios/{id}:
    get:
      summary: Obtener usuario por ID
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
      responses:
        '200':
          description: Usuario encontrado
components:
  schemas:
    Usuario:
      type: object
      properties:
        id:
          type: integer
        nombre:
          type: string
        email:
          type: string
```

👉 Esto genera automáticamente una documentación completa con ejemplos de peticiones y respuestas.

7 Swagger en frameworks populares

FastAPI (Python)

- Swagger UI y ReDoc se generan automáticamente en: `http://localhost:8000/docs` (Swagger) y `http://localhost:8000/redoc` (ReDoc).

Spring Boot (Java)

- Usando la librería springdoc-openapi: la doc se genera en `http://localhost:8080/swagger-ui.html`.

Express (Node.js)

- Con el paquete swagger-ui-express y un archivo YAML/JSON.

8 Ejemplo práctico con FastAPI

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Producto(BaseModel):
    id: int
    nombre: str
    precio: float

@app.get("/productos", response_model=list[Producto])
def listar_productos():
    return [
        {"id": 1, "nombre": "Notebook", "precio": 1500},
        {"id": 2, "nombre": "Tablet", "precio": 900}
    ]
```

👉 Al ejecutar la app, FastAPI genera automáticamente documentación en Swagger UI (`/docs`) y en ReDoc (`/redoc`).

9 Buenas prácticas en la documentación automática

- Definir **ejemplos** de request y response: ayuda a los desarrolladores a entender cómo interactuar.
- Incluir descripciones claras para cada endpoint y campo.
- Documentar **errores posibles**, no solo los casos de éxito.
- Actualizar la especificación cada vez que cambie la API.
- Versionar la documentación: `/api/v1/docs`, `/api/v2/docs`.

10 Conclusión del capítulo

OpenAPI es el estándar actual para describir APIs REST. **Swagger** es el ecosistema de herramientas más usado para generar documentación interactiva.

Una API documentada con Swagger permite que cualquier desarrollador la entienda y la pruebe fácilmente.

La documentación automática convierte a la API en algo profesional, transparente y usable.

👉 En resumen: si tu API no tiene documentación automática, está incompleta.

Versionado de APIs (v1, v2 en endpoints)

1 Introducción

Con el tiempo, toda API necesita evolucionar:

- Se agregan nuevas funcionalidades.
- Se cambian estructuras de datos.
- Se eliminan campos obsoletos.

👉 El problema: ¿cómo mejorar la API sin romper a los clientes que ya la usan? La solución es el **versionado de APIs**, que permite mantener varias versiones en paralelo.

2 ¿Qué es el versionado de APIs?

El versionado es la práctica de asignar un número de versión a la API (ejemplo: `v1`, `v2`) para indicar qué conjunto de reglas, endpoints y respuestas se aplica.

Beneficios

- Compatibilidad hacia atrás: los clientes actuales pueden seguir usando la versión vieja.
- Facilita la evolución de la API sin interrumpir servicios.
- Ofrece un camino controlado para que los clientes migren a la nueva versión.

3 Estrategias comunes de versionado

Existen varias formas de implementar versiones en APIs REST:

1) Versión en la URL (la más usada)

```
/api/v1/usuarios
/api/v2/usuarios
```

Ventajas: claro y fácil de entender; compatible con navegadores, Postman y cURL.

Desventajas: la URL cambia con cada versión, lo que puede generar duplicación de código.

2) Versión en el header

```
GET /usuarios
Accept: application/json; version=2
```

Ventajas: la URL permanece igual; el cliente decide la versión mediante un header.

Desventajas: menos visible a simple vista; más difícil de probar directamente en el navegador.

3) Versión en el parámetro de query

```
GET /usuarios?version=2
```

Ventajas: fácil de implementar.

Desventajas: no es estándar; puede generar confusión si se mezclan con otros query params.

4 Ejemplo práctico de versionado

Versión 1 (v1)

```
GET /api/v1/usuarios/1

Respuesta:
{
  "id": 1,
  "nombre": "Ana"
}
```

Versión 2 (v2)

```
GET /api/v2/usuarios/1

Respuesta:
{
  "id": 1,
  "nombre": "Ana",
  "email": "ana@ejemplo.com",
  "telefono": "+54 351 1234567"
}
```

👉 La v2 agrega nuevos campos (`email`, `telefono`) sin romper la v1.

5 Migración entre versiones

Cuando se lanza una nueva versión:

- Mantener la versión anterior activa por un tiempo.
- Notificar a los clientes de los cambios (changelog).
- Dar un periodo de transición para migrar.
- Deprecar la versión vieja solo cuando la mayoría ya migró.

Ejemplo de mensaje de deprecación

```
{
  "aviso": "La versión v1 quedará obsoleta en diciembre 2025. Por favor migre a
v2."
}
```

6 Buenas prácticas en versionado

- Incluir siempre la versión desde el inicio de la API (`v1`) para evitar problemas futuros.
- Usar URLs claras cuando se opte por versionado en la ruta.
- Documentar bien los cambios entre versiones (Swagger / OpenAPI o changelog).

- Evitar romper compatibilidad: no cambiar el significado de campos existentes.
- Agregar nuevos campos sin eliminar los viejos inmediatamente.
- Definir políticas de deprecación claras: cuánto tiempo se mantiene una versión vieja.

7 Ejemplo en frameworks

FastAPI (Python)

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/api/v1/usuarios/{id}")
def get_usuario_v1(id: int):
    return {"id": id, "nombre": "Ana"}

@app.get("/api/v2/usuarios/{id}")
def get_usuario_v2(id: int):
    return {"id": id, "nombre": "Ana", "email": "ana@ejemplo.com"}
```

Express (Node.js)

```
const express = require("express");
const app = express();

app.get("/api/v1/usuarios/:id", (req, res) => {
  res.json({ id: req.params.id, nombre: "Ana" });
});

app.get("/api/v2/usuarios/:id", (req, res) => {
  res.json({ id: req.params.id, nombre: "Ana", email: "ana@ejemplo.com" });
});

app.listen(3000, () => console.log("Servidor en http://localhost:3000"));
```

👉 Cada versión se mantiene independiente, permitiendo cambios sin romper la anterior.

8 Conclusión del capítulo

El versionado de APIs es necesario para permitir la evolución sin romper integraciones existentes.

- La forma más común es poner la versión en la URL (`/api/v1`, `/api/v2`).
- También existen enfoques con headers o query params.
- Buenas prácticas: mantener compatibilidad, documentar cambios, dar tiempo de transición, definir políticas de deprecación.

👉 En resumen: el versionado es la clave para APIs REST duraderas y confiables.

Buenas prácticas de diseño de APIs REST

1 Introducción

Una API REST puede estar técnicamente correcta y aun así ser difícil de usar si no sigue buenas prácticas de diseño.

- Una API mal diseñada genera frustración en los clientes y aumenta el costo de mantenimiento.
- Una API bien diseñada es intuitiva, predecible y consistente, lo que la convierte en un producto profesional.

👉 En este capítulo veremos las mejores prácticas para diseñar APIs REST limpias y efectivas.

2 Consistencia en endpoints y rutas

Usar sustantivos, no verbos

```
/crearUsuario   # incorrecto
/usuarios      # correcto
```

👉 El método HTTP ya define la acción (`POST /usuarios` crea un usuario).

Usar plural para colecciones

```
/usuario   # incorrecto
/usuarios  # correcto
```

Usar minúsculas y guiones medios

```
/HistorialCompras   # incorrecto
/historial-compras # correcto
```

Mantener jerarquía clara

```
/usuarios/1/pedidos       # pedidos del usuario 1
/productos/15/comentarios # comentarios del producto 15
```

3 Uso correcto de métodos HTTP

Cada método debe usarse de acuerdo con su semántica:

- **GET**: obtener información (nunca modificar).
- **POST**: crear un recurso nuevo.
- **PUT**: actualizar un recurso completo.
- **PATCH**: actualizar parcialmente un recurso.
- **DELETE**: eliminar un recurso.

👉 Esto evita confusión y mantiene la API predecible.

4 Uso adecuado de códigos de estado

Siempre devolver códigos HTTP correctos:

- `200 OK`: operación exitosa.
- `201 Created`: recurso creado.
- `204 No Content`: eliminación sin datos.
- `400 Bad Request`: error en la petición del cliente.
- `401 Unauthorized`: falta autenticación.
- `403 Forbidden`: no tiene permisos.
- `404 Not Found`: recurso no existe.
- `500 Internal Server Error`: error inesperado del servidor.

👉 Nunca devolver siempre `200`, incluso en errores.

5 Estructura clara en respuestas JSON

- Siempre usar JSON válido y consistente.
- Mantener nombres de campos en minúsculas y `snake_case` o `camelCase` (elegir uno y no mezclar).
- Responder con objetos bien estructurados.

Ejemplo correcto

```
{
  "id": 1,
  "nombre": "Ana",
  "email": "ana@ejemplo.com"
}
```

Ejemplo incorrecto

```
["Ana", "ana@ejemplo.com"]  # difícil de entender
```

👉 Además, incluir mensajes claros en errores:

```
{
  "error": "Bad Request",
  "mensaje": "El campo 'email' es obligatorio"
}
```

6 Paginación y filtros

Cuando una colección puede devolver miles de resultados, paginación y filtros son indispensables.

Ejemplo

```
GET /usuarios?pagina=2&limite=20
```

Respuesta

```
{
  "pagina": 2,
  "limite": 20,
  "total": 95,
  "usuarios": [
    { "id": 21, "nombre": "Luis" },
    { "id": 22, "nombre": "Laura" }
  ]
}
```

👉 Esto mejora el rendimiento y evita respuestas demasiado grandes.

7 Autenticación y seguridad

- Usar HTTPS siempre.
- No pasar credenciales en la URL.
- Usar tokens (API Keys, JWT).
- Controlar permisos por roles.
- Evitar exponer información sensible en errores (`500` debe ser genérico).

👉 Una API sin seguridad es una puerta abierta a ataques.

8 Versionado claro

Siempre versionar la API desde el inicio:

```
/api/v1/usuarios
/api/v2/usuarios
```

👉 Permite evolucionar sin romper compatibilidad con clientes antiguos.

9 Documentación automática

- Incluir documentación con OpenAPI/Swagger.
- Documentar ejemplos de requests y responses.
- Incluir lista de errores posibles para cada endpoint.

👉 La documentación es parte del producto, no un accesorio.

10 Estándar en mensajes de error

Definir un formato único para errores. Ejemplo:

```
{
  "error": "Not Found",
  "mensaje": "El producto con id 99 no existe",
  "codigo": 404,
  "timestamp": "2025-09-10T12:30:00Z",
  "ruta": "/productos/99"
}
```

👉 Esto ayuda a los clientes a interpretar los fallos de manera consistente.

11 Estandarizar convenciones de nombres

- **Campos booleanos**: prefijo `is_` o `activo`. Ejemplo: `"activo": true`.
- **Fechas y horas**: siempre en formato ISO 8601 (`2025-09-10T12:30:00Z`).
- **Monedas y precios**: incluir decimales y, si es necesario, el código de moneda. Ejemplo: `"precio": 1500.00, "moneda": "USD"`.

12 Ejemplo de una API bien diseñada

Endpoint

```
GET /api/v1/productos?categoria=electronica&pagina=1&limite=2
```

Respuesta

```
{
  "pagina": 1,
  "limite": 2,
  "total": 25,
  "productos": [
    { "id": 101, "nombre": "Notebook Gamer", "precio": 2200.00, "stock": true },
    { "id": 102, "nombre": "Auriculares Bluetooth", "precio": 150.00, "stock":
false }
  ]
}
```

👉 Claridad en la ruta, paginación, nombres consistentes y uso correcto de JSON.

13 Conclusión del capítulo

Las buenas prácticas en diseño de APIs REST hacen la diferencia entre una API usable y una API profesional, confiable y escalable.

- Endpoints claros y consistentes.
- Uso correcto de métodos HTTP y códigos de estado.
- Respuestas JSON bien estructuradas.
- Manejo estándar de errores.
- Seguridad (HTTPS, tokens).
- Versionado y documentación clara.

👉 En resumen: una API bien diseñada habla el mismo idioma que los desarrolladores.

Testing de APIs REST (Postman, pytest, herramientas automatizadas)

1 Introducción

El testing de APIs REST es el proceso de verificar que los endpoints:

- Devuelven las respuestas correctas.
- Manejan errores de forma adecuada.
- Respetan las reglas de negocio.
- Se mantienen estables a medida que cambia el código.

👉 Una API no probada puede parecer que funciona en casos simples, pero fallar en producción con datos reales.

2 Tipos de pruebas en APIs REST

Pruebas manuales
Usando herramientas como Postman o navegadores. Útiles en etapas iniciales, pero no escalables.

Pruebas automatizadas
Scripts que ejecutan peticiones y validan las respuestas automáticamente. Se integran en pipelines de CI/CD.

Pruebas unitarias
Verifican funciones pequeñas (ej. serialización de JSON).

Pruebas de integración
Verifican cómo interactúan varios módulos de la API.

Pruebas de contrato
Validan que la API cumpla con lo definido en la especificación (ej. OpenAPI).

Pruebas de carga
Simulan múltiples usuarios para ver si la API soporta tráfico intenso.

3 Testing con Postman

Postman es la herramienta más popular para probar APIs manualmente y también soporta tests automáticos.

Uso básico (manual)

- Crear una colección con los endpoints de la API.
- Configurar el método (GET, POST, etc.), la URL, headers y body.
- Ejecutar la petición y ver la respuesta.

Ejemplo de test automático en Postman (pestaña Tests, lenguaje JavaScript)

```
pm.test("El status debe ser 200", function () {
  pm.response.to.have.status(200);
});

pm.test("La respuesta debe ser JSON", function () {
  pm.response.to.have.header("Content-Type", "application/json; charset=utf-8");
});

pm.test("El campo 'nombre' debe existir", function () {
  var jsonData = pm.response.json();
  pm.expect(jsonData).to.have.property("nombre");
});
```

👉 Así, cada vez que se ejecute el request, Postman validará automáticamente las condiciones.

4 Testing con pytest (Python)

Para proyectos en Python, se suele usar pytest junto con requests o directamente con FastAPI TestClient.

Ejemplo usando requests + pytest

```
import requests

def test_obtener_usuario():
    resp = requests.get("https://jsonplaceholder.typicode.com/users/1")
    assert resp.status_code == 200
    data = resp.json()
    assert "name" in data
    assert data["id"] == 1
```

👉 Este test verifica que:

- El endpoint `/users/1` devuelve status 200.
- La respuesta incluye un campo `name`.
- El id del usuario sea 1.

Fuente de datos: JSONPlaceholder.

Ejemplo con FastAPI TestClient

```
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_crear_producto():
    resp = client.post("/productos", json={"nombre": "Tablet", "precio": 900})
    assert resp.status_code == 201
    data = resp.json()
    assert data["nombre"] == "Tablet"
    assert data["precio"] == 900
```

👉 Aquí no se necesita servidor externo, se prueba directamente la app en memoria.

5 Herramientas automatizadas adicionales

Newman (CLI de Postman): permite ejecutar colecciones desde la terminal. Ideal para integrarlo en CI/CD (Jenkins, GitHub Actions).

```
newman run coleccion.json
```

Pytest + CI/CD: correr tests automáticamente en pipelines (por ejemplo, GitHub Actions ejecuta pytest en cada push).

Herramientas de carga (Load Testing): JMeter, k6, Locust simulan usuarios concurrentes para medir rendimiento.

Ejemplo con k6 (JavaScript)

```
import http from "k6/http";
import { check } from "k6";

export default function () {
  let res = http.get("https://api.ejemplo.com/usuarios");
  check(res, { "status 200": (r) => r.status === 200 });
}
```

6 Ejemplo de flujo completo de testing

1. Definir casos de prueba: `/usuarios` devuelve lista (200, JSON válido); `/usuarios/99` devuelve 404; `/usuarios` con JSON mal formado devuelve 400.
2. Probar manualmente en Postman para confirmar los endpoints.
3. Automatizar con Postman + Newman para ejecutar las colecciones en cada build.
4. Agregar pytest (o equivalente) para pruebas unitarias e integración.
5. Ejecutar pruebas de carga para verificar que la API soporte la cantidad esperada de usuarios.

7 Buenas prácticas en testing de APIs REST

- Probar tanto casos de éxito como de error (por ejemplo, crear un producto con precio negativo debe devolver 400).
- Usar datos realistas en las pruebas; evitar ejemplos triviales.
- Automatizar siempre que sea posible: los tests manuales ayudan al inicio, pero los automatizados garantizan estabilidad a largo plazo.
- Incluir pruebas de seguridad: endpoints protegidos deben devolver `401` si no hay token.
- Integrar tests en CI/CD para correrlos en cada despliegue.

8 Conclusión del capítulo

El testing de APIs REST asegura que los endpoints funcionen como se espera y se mantengan estables con el tiempo.

- Herramientas como Postman sirven tanto para pruebas manuales como automatizadas.
- pytest (Python) y librerías equivalentes en otros lenguajes permiten pruebas más integradas.
- Newman, JMeter, k6 y Locust son útiles para pruebas automáticas y de rendimiento.

👉 En resumen: una API sin tests es una API insegura e incompleta.

Consumo de APIs REST desde Python (requests, httpx)

1 Introducción

Python es uno de los lenguajes más usados para interactuar con APIs REST, tanto en:

- Automatización (scripts que consultan APIs de terceros).
- Backend (servicios que consumen APIs externas).
- Data science (descargar datos de APIs para análisis).

👉 Las dos librerías más comunes son: requests (sencilla y estable) y httpx (moderna, con soporte de asincronía).

2 Consumo de APIs con requests

Instalación

```
pip install requests
```

Ejemplo 1 — GET básico

```
import requests

resp = requests.get("https://jsonplaceholder.typicode.com/users/1")

if resp.status_code == 200:
    data = resp.json()
    print("Usuario:", data["name"], "-", data["email"])
else:
    print("Error:", resp.status_code)
```

👉 Flujo: petición → verificar código → convertir a JSON → procesar datos.

Ejemplo 2 — POST con JSON

```
import requests
```

```
nuevo_post = {
    "title": "Nuevo Post",
    "body": "Contenido de ejemplo",
    "userId": 1
}

resp = requests.post("https://jsonplaceholder.typicode.com/posts",
json=nuevo_post)

print("Status:", resp.status_code)
print("Respuesta:", resp.json())
```

👉 Con el parámetro `json=`, requests convierte automáticamente el objeto Python en JSON.

Ejemplo 3 — PUT (actualizar recurso completo)

```
import requests

actualizar = {
    "id": 1,
    "title": "Título actualizado",
    "body": "Contenido modificado",
    "userId": 1
}

resp = requests.put("https://jsonplaceholder.typicode.com/posts/1",
json=actualizar)
print("Respuesta:", resp.json())
```

Ejemplo 4 — PATCH (actualización parcial)

```
import requests

cambio = { "title": "Título con PATCH" }

resp = requests.patch("https://jsonplaceholder.typicode.com/posts/1",
json=cambio)
print("Respuesta:", resp.json())
```

Ejemplo 5 — DELETE

```
import requests

resp = requests.delete("https://jsonplaceholder.typicode.com/posts/1")
print("Código de estado:", resp.status_code)
```

3 Manejo de headers personalizados

En muchas APIs, hay que enviar tokens de autenticación en el header.

```
import requests

headers = {
    "Authorization": "Bearer 12345",
```

```
    "Accept": "application/json"
}

resp = requests.get("https://api.ejemplo.com/pedidos", headers=headers)
print(resp.status_code, resp.json())
```

4 Consumo de APIs con httpx

Instalación

```
pip install httpx
```

Características principales

- Similar a requests, pero con soporte para `async/await`.
- Mejor rendimiento en aplicaciones que hacen muchas peticiones concurrentes.

Ejemplo 1 — GET con httpx (síncrono)

```
import httpx

resp = httpx.get("https://jsonplaceholder.typicode.com/users/1")
print(resp.status_code, resp.json())
```

Ejemplo 2 — GET con httpx (asíncrono)

```
import httpx
import asyncio

async def main():
    async with httpx.AsyncClient() as client:
        resp = await client.get("https://jsonplaceholder.typicode.com/users")
        print(resp.status_code)
        print(resp.json())

asyncio.run(main())
```

👉 Ideal cuando hay que consultar varias APIs en paralelo sin bloquear el programa.

Ejemplo 3 — POST con httpx

```
import httpx

nuevo_usuario = { "nombre": "Laura", "email": "laura@ejemplo.com" }

resp = httpx.post("https://jsonplaceholder.typicode.com/users",
json=nuevo_usuario)
print(resp.status_code, resp.json())
```

Ejemplo 4 — Headers en httpx

```
import httpx
```

```
headers = {
    "Authorization": "Bearer 12345",
    "Accept": "application/json"
}

resp = httpx.get("https://api.ejemplo.com/datos", headers=headers)
print(resp.json())
```

5 Manejo de errores en Python

Tanto con requests como con httpx es importante capturar excepciones:

```
import requests

try:
    resp = requests.get("https://api.ejemplo.com/datos", timeout=5)
    resp.raise_for_status()  # Lanza error si no es 200-299
    print(resp.json())
except requests.exceptions.Timeout:
    print("La petición tardó demasiado")
except requests.exceptions.HTTPError as err:
    print("Error HTTP:", err)
except Exception as e:
    print("Error general:", e)
```

👉 Esto evita que el programa se rompa si la API falla.

6 Ejemplo práctico: consultar una lista de productos

```
import requests

url = "https://jsonplaceholder.typicode.com/posts"

resp = requests.get(url)
if resp.status_code == 200:
    productos = resp.json()
    for p in productos[:5]:
        print(f"ID: {p['id']} - Título: {p['title']}")
else:
    print("Error al obtener productos:", resp.status_code)
```

👉 Simulación de obtener productos de una tienda usando JSONPlaceholder.

7 ¿Cuándo usar requests y cuándo httpx?

requests

- Ideal para scripts simples y proyectos pequeños.
- Muy estable y ampliamente soportado.

httpx

- Ideal para aplicaciones modernas con alta concurrencia.

- Soporta HTTP/2, timeouts más precisos y asincronía.

8 Buenas prácticas en Python para consumo de APIs

- Usar `timeout` para evitar bloqueos (`timeout=5`).
- Manejar errores con `try/except`.
- Reutilizar clientes (`Session` en requests, `AsyncClient` en httpx) en lugar de abrir conexión cada vez.
- Guardar tokens y credenciales en variables de entorno (no en el código).
- Validar siempre la estructura de la respuesta antes de usarla.

9 Conclusión del capítulo

Python es excelente para consumir APIs REST gracias a librerías como requests y httpx.

- **requests** es simple y confiable para la mayoría de los casos.
- **httpx** aporta asincronía y mayor rendimiento en aplicaciones modernas.
- Es fundamental manejar headers, errores y timeouts correctamente.

👉 En resumen: con Python, consumir APIs REST es rápido, seguro y flexible.

Comparación de APIs REST con otras arquitecturas (SOAP, GraphQL)

1 Introducción

Las APIs son la forma en que los sistemas intercambian datos. Aunque REST con JSON es hoy el estándar más usado, no es la única arquitectura:

- Antes fue común SOAP (Simple Object Access Protocol).
- Hoy crece el uso de GraphQL como alternativa moderna.

👉 Conocer sus diferencias permite elegir la herramienta adecuada para cada proyecto.

2 APIs REST

- Basadas en el protocolo HTTP.
- Usan métodos estándar (GET, POST, PUT, DELETE, etc.).
- Intercambian datos generalmente en JSON.
- Los recursos se representan como URLs (`/usuarios/1`).

Ejemplo REST

```
GET /usuarios/1
```

Respuesta

```
{
  "id": 1,
  "nombre": "Ana",
  "email": "ana@ejemplo.com"
}
```

Ventajas

- Simple y fácil de entender.
- Muy usado y compatible en casi todos los lenguajes.
- JSON es ligero y legible.

Desventajas

- *Overfetching*: a veces devuelve más datos de los necesarios.
- *Underfetching*: a veces hay que hacer varias peticiones para obtener todo lo necesario.

3 APIs SOAP

SOAP (Simple Object Access Protocol) es un estándar más antiguo.

- Basado en XML.
- Opera sobre HTTP pero también sobre otros protocolos (SMTP, TCP).
- Muy usado en servicios empresariales y en sistemas legados (bancos, seguros, gobiernos).

Ejemplo SOAP

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
  <soapenv:Header/>
  <soapenv:Body>
    <getUsuarioRequest>
      <id>1</id>
    </getUsuarioRequest>
  </soapenv:Body>
</soapenv:Envelope>
```

Respuesta SOAP

```
<soapenv:Envelope>
  <soapenv:Body>
    <getUsuarioResponse>
      <usuario>
        <id>1</id>
        <nombre>Ana</nombre>
        <email>ana@ejemplo.com</email>
      </usuario>
    </getUsuarioResponse>
  </soapenv:Body>
</soapenv:Envelope>
```

Ventajas

- Tipado fuerte y contrato formal usando WSDL (Web Services Description Language).
- Estándares robustos para seguridad (WS-Security) y transacciones distribuidas.
- Fiable en entornos corporativos críticos.

Desventajas

- XML es más pesado y verboso que JSON.
- Complejidad en la implementación.
- Menos flexible que REST y GraphQL.

4 APIs GraphQL

- Creado por Facebook en 2015 como alternativa a REST.
- El cliente define exactamente qué datos necesita, evitando *overfetching* y *underfetching*.
- Usa un único endpoint (ejemplo: `/graphql`).
- Los datos se consultan con un lenguaje de queries similar a SQL.

Ejemplo GraphQL (consulta)

```
{
  usuario(id: 1) {
    nombre
    email
    pedidos {
      id
      total
    }
  }
}
```

Respuesta GraphQL

```
{
  "data": {
    "usuario": {
      "nombre": "Ana",
      "email": "ana@ejemplo.com",
      "pedidos": [
        { "id": 100, "total": 1500 },
        { "id": 101, "total": 900 }
      ]
    }
  }
}
```

Ventajas

- El cliente pide exactamente lo que necesita.
- Reduce la cantidad de llamadas a la API.
- Muy eficiente en aplicaciones con muchas relaciones (redes sociales).
- Autodocumentación y tipado estricto (schema).

Desventajas

- Requiere más esfuerzo en la configuración del servidor.
- Puede generar consultas muy complejas y pesadas.
- No siempre es la mejor opción para APIs simples.

5 Comparación REST vs SOAP vs GraphQL

- **Formato de datos**: REST → JSON (a veces XML); SOAP → solo XML; GraphQL → JSON.
- **Complejidad**: REST = simple; SOAP = complejo; GraphQL = medio.
- **Contrato**: REST = no estricto; SOAP = WSDL estricto; GraphQL = schema con tipado fuerte.
- **Flexibilidad**: REST = limitada; SOAP = baja; GraphQL = muy alta.
- **Seguridad**: REST = HTTPS, JWT; SOAP = WS-Security, SSL; GraphQL = HTTPS, JWT.
- **Overfetching/Underfetching**: REST = sí, posible; SOAP = menos probable (contrato fijo); GraphQL = no, el cliente define.
- **Casos de uso**: REST = web, móviles, microservicios; SOAP = entornos corporativos, banca, seguros; GraphQL = aplicaciones modernas y frontends dinámicos.

6 ¿Cuándo usar cada arquitectura?

REST

- La mejor opción por defecto.
- Ideal para APIs públicas, móviles, microservicios.
- Fácil de aprender y de usar.

SOAP

- Cuando se necesitan transacciones seguras y estándares corporativos.
- En sistemas bancarios, seguros, e-government.
- Útil si ya existe infraestructura basada en SOAP.

GraphQL

- Cuando el frontend necesita flexibilidad en la obtención de datos.
- Ideal para aplicaciones con relaciones complejas (redes sociales, dashboards).
- Cuando se busca reducir el número de requests al servidor.

7 Conclusión del capítulo

- **REST**: estándar actual, simple y eficiente.
- **SOAP**: legado robusto, usado en entornos críticos con contratos estrictos.
- **GraphQL**: moderno, flexible y potente, evita problemas de REST en consultas complejas.

👉 En resumen: usa REST como opción general; considera SOAP si trabajás en sistemas legados empresariales; considera GraphQL si tu aplicación necesita gran flexibilidad y optimización en consultas.